

Overview of the Simulation-Free Score and Flow Matching Objective

We consider stochastic processes that can be represented by SDEs of the form:

$$dx_t = u_t(x_t)dt + g(t)dB_t$$

where $u_t : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ a smooth vector field and B_t the standard Brownian motion on $\mathbb{R}^d$.

Let $\mathcal{P}$ be the space of probability measures on the path space $C([0, 1], \mathbb{R}^d)$. Assume that the measure $P \in \mathcal{P}$ admits density p with respect to the Lebesgue measure. A density $p(x_0)$ evolved according to an SDE induces marginal distributions $p_t(x_t)$, viewed as a function $p : [0, 1] \times \mathbb{R} \rightarrow \mathbb{R}_+$. They are characterized by the initial condition p_0 and the Fokker-Planck equation:

$$\partial_t p_t = -\nabla \cdot (p_t u_t) + \frac{g^2(t)}{2} \Delta p_t$$

where $\Delta p_t = \nabla \cdot (\nabla p_t)$ is the Laplacian.

In [Simulation-Free Schrödinger Bridges via Score and Flow Matching](#) (among others) it is shown that the SDE above admits an ODE formulation generating the exact same marginals:

$$dx_t = u_t^o(x_t)dt, \quad \partial_t p_t = -\nabla \cdot (p_t u_t^o)$$

where the probability flow ODE vector field u_t^o is defined by:

$$u_t^o(x) = u_t(x) - \frac{g^2(t)}{2} \nabla \log p_t(x)$$

Therefore, the unknown drift of the target SDE can be decomposed into a deterministic vector field u_t^o and a score function $\nabla \log p_t$:

$$u_t(x) = u_t^o(x) + \frac{g^2(t)}{2} \nabla \log p_t(x)$$

Under mild regularity conditions, the ODE $dx_t = u_t^o(x_t)dt$, generates the time marginals of P , and the SDE $dx_t = [u_t^o(x_t) + 1/2g^2(t)\nabla \log p_t(x_t)]dt + g(t)dB_t$, generates the Markovization of P (see [Simulation-Free Schrödinger Bridges via Score and Flow Matching](#)).

By isolating these components, we can independently model the probability flow ODE drift u_t^o and the score $\nabla \log p_t$ using neural networks, which can be trained by the following (unconditional) score and flow matching objective:

$$L_{U[SF]^2M}(\theta) = \mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{x_t \sim p_t(\cdot)} [\|u_\theta(t, x) - u_t^o(x)\|^2 + \lambda^2(t) \|s_\theta(t, x) - \nabla \log p_t(x)\|^2]$$

Once trained u_θ , s_θ can be used to simulate the SDE from the source samples x_0 using the algorithm bellow:

Algorithm 1: Simulation-Free Score and Flow Matching Inference

Input: Source distribution q_0 , flow and score networks u_θ , s_θ , diffusion schedule $g(t)$, integration step size Δt

Sample $x_0 \sim q_0(\cdot)$ $N \leftarrow 1/\Delta t$

for $k = 0$ **to** $N - 1$ **do**

$t_k \leftarrow k\Delta t$
 $u_k \leftarrow u_\theta(k, x_k) + \frac{g^2(t_k)}{2}s_\theta(k, x_k)$
 $x_{k+1} \leftarrow N(x_k + u_k\Delta t, g^2(t_k)\Delta t\mathbf{I})$

Output: Samples x_1

Let $P \in \mathcal{P}$ a probability measure with density $p(x)$ and time marginals $p_t(x)$. Let $z \sim q(\cdot)$ be a latent variable with density $q(\cdot)$. Then P can be disintegrated as:

$$P(\cdot) = \int_{\mathbb{R}^d} P(\cdot | z)q(dz)$$

Its density can also be disintegrated as follows:

$$p(\cdot) = \int_{\mathbb{R}^d} p(\cdot | z)q(dz)$$

where the conditional density $p_t(\cdot | z)$ is induced by the following conditional SDE:

$$dx_t = u_t(x_t | z)dt + g(t)dB_t$$

We denote $u_t^o(\cdot | z)$ as the deterministic conditional vector field and $\nabla \log p_t(\cdot | z)$ as the conditional score function. Then it is shown that the probability ODE flow and the score that generates the marginals of the process P are given by:

$$u_t^o(x) = \mathbb{E}_{z \sim q(\cdot)} \left[\frac{u_t^o(x | z)p_t(x | z)}{p_t(x)} \right]$$

$$\nabla \log p_t(x) = \mathbb{E}_{z \sim q(\cdot)} \left[\frac{p_t(x | z)}{p_t(x)} \nabla \log p_t(x | z) \right]$$

Therefore we can define the conditional simulation-free score and flow matching objective:

$$L_{[SF]^2M}(\theta) = \mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{z \sim q(\cdot), x \sim p_t(\cdot | z)} [\|u_\theta(t, x) - u_t^o(x | z)\|^2 + \lambda^2(t) \|s_\theta(t, x) - \nabla \log p_t(x | z)\|^2]$$

It can be shown that if $p_t(x)$, for all $x \in \mathbb{R}^d$, and $t \in [0, 1]$ then the conditional and unconditional objectives are equivalent, i.e., $\nabla_\theta L_{U[SF]^2M}(\theta) = \nabla_\theta L_{[SF]^2M}(\theta)$.

We consider $z = (x_0, x_1)$ and defining the conditional time marginal $p_t(x | z)$ to be the Brownian Bridge with constant diffusion scale σ . In that case the conditional ODE vector field $u_t^o(x | z)$ and the conditional score function $\nabla \log p_t(x | z)$ are known analytically:

$$u_t^o(x | (x_0, x_1)) = \frac{1 - 2t}{2t(1 - t)}(x - (tx_1 + (1 - t)x_0)) + (x_1 - x_0)$$

$$\nabla \log p_t(x | (x_0, x_1)) = \frac{tx_1 + (1 - t)x_0 - x}{\sigma^2 t(1 - t)}$$

Let the reference process Q of the Schrödinger Problem be a Brownian motion. Then the Schrödinger Bridge P^* is a mixture of Brownian bridges weighted by the entropic optimal transport plan $\pi_{2\sigma^2}^*$:

$$P^*(\cdot) = \int_{\mathbb{R}^d \times \mathbb{R}^d} Q(\cdot | x_0, x_1) \pi_{2\sigma^2}^*(dx_0, dx_1)$$

If u_θ^* , s_θ^* are the globally minimizers of $L_{[SF]^2 M}$ with coupling $(x_0, x_1) \sim \pi_{2\sigma^2}^*(\cdot, \cdot)$, then P^* is defined by the SDE with drift $[u_\theta^* + 1/2g^2(t)s_\theta^*]$, and diffusion $g(t)$.

But the real optimal coupling π^* is intractable and we can only approximate it by computing the EOT plan between the empirical distributions q_0, q_1 , using the Sinkhorn algorithm, which has $\mathcal{O}(n^2)$ computational complexity.

If the cost is high, then the plan can be further approximated using minibatch OT.

Algorithm 2: Score and Flow Matching Training

Input: Efficiently samplable $q(\cdot)$, $p_t(\cdot | z)$, computable $u_t^o(\cdot | z)$, initial networks u_θ, s_θ

while *Training* **do**

$t \sim U[0, 1]; z \sim q(\cdot); x \sim p_t(\cdot | z)$
 $\theta^* = \arg \min_\theta \{ \|u_\theta(t, x) - u_t^o(x | z)\|^2 + \lambda^2(t) \|s_\theta(t, x) - \nabla \log p_t(x | z)\|^2 \}$

Output: $u_{\theta^*}, s_{\theta^*}$

Algorithm 3: Looped Minibatch Simulation-Free Score and Flow Matching Training

Input: Samplable q_0, q_1 , number of outer loop iterations L , number of inner loop iterations I , cache size n , noise term σ , weighting schedule $\lambda(t)$, initial networks u_θ, s_θ

```

for  $l = 0$  to  $L - 1$  do
  for  $i = 0$  to  $I - 1$  do
    if  $l = 0$  then
       $x_0, x_1 \sim q_0^{\otimes m}, q_1^{\otimes m}$  (sample batches independently)
       $\pi \leftarrow \text{Sinkhorn}(x_0, x_1, 2\sigma^2)$ 
       $x_0, x_1 \sim \pi^{\otimes m}$ 
    else
       $x_0, x_1 \sim \mathbb{T}^{\otimes m}$  (sample from cache)
       $t \sim U[0, 1]$ 
       $p_t(x \mid x_0, x_1) \leftarrow N(x; tx_1 + (1 - t)x_0, \sigma^2 t(1 - t))$ 
       $x \sim p_t(x \mid x_0, x_1)$ 
       $\theta^* = \arg \min_\theta \{ \|u_\theta(t, x) - u_t^o(x \mid x_0, x_1)\|^2 + \lambda^2(t) \|s_\theta(t, x) - \nabla \log p_t(x \mid x_0, x_1)\|^2 \}$ 
       $\mathbb{T}_f \leftarrow (x_0, \hat{x}_1)^{\otimes \lfloor n/2 \rfloor}$  (generate independent forward-simulated trajectories to build a cache size  $n/2$ , according to Algorithm 1)
       $\mathbb{T}_b \leftarrow (\hat{x}_0, x_1)^{\otimes \lfloor n/2 \rfloor}$  (where  $\hat{x}_1$  is sampled according to the backwards analog of Algorithm 1)
       $\mathbb{T} \leftarrow [\mathbb{T}_f, \mathbb{T}_b]$ 

```

Output: $u_{\theta^*}, s_{\theta^*}$

Algorithmic Implementation

Simulating Trajectories to Build the Cache (Outer Loop)

In the looped version of the algorithm, after updating the network parameters in the inner loop, we must generate simulated trajectories to build the cache $\mathbb{T}$ for the next iteration.

The forward cache $\mathbb{T}_f$ is generated by sampling initial points x_0 and simulating the forward SDE using Euler-Maruyama integration. The discrete update for step size Δt is:

$$x_{k+1} \sim \mathcal{N}(x_k + u_k \Delta t, g^2(t_k) \Delta t \mathbf{I})$$

where the combined forward drift is evaluated as $u_k = u_\theta(t_k, x_k) + \frac{g^2(t_k)}{2}s_\theta(t_k, x_k)$.

The backward cache $\mathbb{T}_b$ is generated by sampling target points x_1 and simulating the reverse-time SDE. According to Anderson's time reversal formula, the backward SDE is given by:

$$dx_t = \left[-u_t^o(x) + \frac{g^2(t)}{2} \nabla \log p_t(x) \right] dt + g(t) dB_t$$

where the formula guarantees that this reversed process induces the same distribution on x_{1-t} as the original SDE does on x_t . Therefore, the combined backward drift is evaluated as $-u_\theta(t_k, x_k) + \frac{g^2(t_k)}{2}s_\theta(t_k, x_k)$. By simulating these forward and backward paths, we populate the empirical data distributions required for the next outer loop.

Solving the Regression Problem (Inner Loop)

During the inner loop, the simulation-free objective simultaneously fits the probability flow ODE drift u_θ and the score function s_θ to match the analytical targets of the conditional Brownian bridge.

Given the source-target pair (x_0, x_1) , the conditional marginal distribution is precisely $p_t(x \mid x_0, x_1) = \mathcal{N}(x \mid \mu_t, \sigma_t^2 \mathbf{I})$, where $\mu_t = tx_1 + (1-t)x_0$ and $\sigma_t = \sigma\sqrt{t(1-t)}$. We generate training samples via the reparameterization trick:

$$x = \mu_t + \sigma_t \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

For a Brownian bridge with constant diffusion scale σ , the exact conditional probability flow ODE drift and the conditional score function are known analytically:

$$u_t^o(x \mid x_0, x_1) = \frac{1-2t}{2t(1-t)}(x - \mu_t) + (x_1 - x_0)$$

$$\nabla \log p_t(x \mid x_0, x_1) = \frac{\mu_t - x}{\sigma^2 t(1-t)}$$

By substituting the reparameterized sample x into these analytical forms, the regression targets reduce strictly to terms of the injected noise ϵ :

$$u_t^o(x \mid x_0, x_1) = \frac{1-2t}{2t(1-t)}(\sigma_t \epsilon) + (x_1 - x_0)$$

$$\nabla \log p_t(x \mid x_0, x_1) = -\frac{\epsilon}{\sigma_t}$$

An by substituting the definition of σ_t we derive:

$$u_t^o(x \mid x_0, x_1) = \frac{1 - 2t}{2\sqrt{t(1-t)}}(\sigma\epsilon) + (x_1 - x_0)$$

$$\nabla \log p_t(x \mid x_0, x_1) = -\frac{\epsilon}{\sigma\sqrt{t(1-t)}}$$

While the original paper proposed scaling the score matching loss by $\lambda(t) = \sigma_t$ to eliminate singularities at $t \rightarrow 0$ and $t \rightarrow 1$, this formulation forces the gradient multiplier to evaluate to zero at the integration boundaries (without resolving the singularity to the flow matching loss), thereby suppressing critical learning dynamics near the source and target distributions. Initial logic and parameters evaluating these diverging boundaries are validated.

We resolve this vanishing gradient problem (and the explosion gradient problem of the flow matching loss) through direct network reparameterization. By structuring the networks to directly output bounded predictions for the deterministic velocity $\hat{v}_\theta(t, x)$ and the standard normal noise $\hat{\epsilon}_\theta(t, x)$, we shift the singular coefficients entirely outside the neural architecture:

$$u_t^o(x \mid x_0, x_1) = \frac{1 - 2t}{2\sqrt{t(1-t)}}\sigma\hat{\epsilon}_\theta(t, x) + \hat{v}_\theta(t, x)$$

$$\nabla \log p_t(x \mid x_0, x_1) = -\frac{\hat{\epsilon}_\theta(t, x)}{\sigma\sqrt{t(1-t)}}$$

The stabilized, unscaled objective is derived by strictly isolating and matching these bounded components:

$$\hat{L}(\theta) = \|\hat{v}_\theta(t, x) - (x_1 - x_0)\|^2 + \|\hat{\epsilon}_\theta(t, x) - \epsilon\|^2$$

The parameters θ are then optimized by evaluating gradient steps directly on this closed-form objective. During inference, the full probability flow ODE drift and score are analytically reconstructed, allowing the combined forward SDE drift to be simplified to $u_k \leftarrow \hat{v}_\theta(t_k, x_k) - \sigma\sqrt{\frac{t_k}{1-t_k}}\hat{\epsilon}_\theta(t_k, x_k)$. That is because from Algorithm 1 we have:

$$u_k \leftarrow u_\theta(t_k, x_k) + \frac{\sigma^2}{2}s_\theta(t_k, x_k)$$

Thus in our parameterization:

$$u_k \leftarrow \left[\hat{v}_\theta(t_k, x_k) + \frac{1 - 2t_k}{2\sqrt{t_k(1-t_k)}}\sigma\hat{\epsilon}_\theta(t_k, x_k) \right] + \frac{\sigma^2}{2} \left[-\frac{\hat{\epsilon}_\theta(t_k, x_k)}{\sigma\sqrt{t_k(1-t_k)}} \right]$$

$$u_k \leftarrow \hat{v}_\theta(t_k, x_k) - \frac{t_k}{\sqrt{t_k(1-t_k)}}\sigma\hat{\epsilon}_\theta(t_k, x_k)$$

Which gives us the derived result. To further guarantee stability, temporal boundary clipping is applied to the time steps prior to integration.

Algorithm 4: Simulation-Free Score and Flow Matching Parameterize Inference

Input: Source distribution q_0 , bounded networks $\hat{v}_\theta, \hat{\epsilon}_\theta$, diffusion scale σ , integration step size Δt

Sample $x_0 \sim q_0(\cdot)$

$N \leftarrow 1/\Delta t$

for $k = 0$ **to** $N - 1$ **do**

$t_k \leftarrow \max(10^{-4}, \min(k\Delta t, 1 - 10^{-4}))$
 $u_k \leftarrow \hat{v}_\theta(t_k, x_k) - \sigma \sqrt{\frac{t_k}{1-t_k}} \hat{\epsilon}_\theta(t_k, x_k)$
 $x_{k+1} \leftarrow x_k + u_k \Delta t + \sigma \sqrt{\Delta t} \mathbf{z} \quad \text{where } \mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$

Output: Samples x_1
